ISIS4426 · Desarrollo de Soluciones Cloud · 2026-20

Entrega 3: aplicación elástica IaaS

Repartición del trabajo, propuesta de despliegue y estado de las correcciones de la Entrega 2

Equipo: Daniela Solarte, Mateo, Samara Martínez y Camila Fajardo · Proyecto GCP `proyecto1-entrega2-desarrollo`
· Documento preparado el 10 de octubre de 2026 · Actualizado el 12 de octubre de 2026 (secciones 8, 9 y 10)

En una página. Las seis correcciones de la Entrega 2 están cerradas y se convierten en la línea base de la Entrega 3. Para levantar varias máquinas de API y de workers sin trabajo manual, la propuesta es **no instalar nada al arrancar**: cada rol (API, worker) tiene una **imagen de máquina propia** con Docker y la imagen del contenedor ya descargada, y un servicio systemd que arranca el contenedor con la versión fijada. El arranque solo lee secretos de Secret Manager y se inicia. El **frontend no usa máquinas**: se publica como archivos estáticos en un bucket detrás del mismo balanceador HTTPS, así comparte dominio con la API y las cookies y el CSRF funcionan sin CORS. Redis (la cola compartida) y ClamAV salen del worker a una **VM de servicios fija**. Esta vez **Samara y Camila corren los escenarios** (Samara: 1 y 3A; Camila: 2 y 3B), con traspaso de Daniela y Mateo, que los corrieron en la Entrega 2.

Contenido

- 1. Estado de partida: correcciones de la Entrega 2 y estado actual de GCP
- 2. Qué exige la Entrega 3
- 3. Propuesta de despliegue eficiente (backend en grupos, frontend sin VM)
- 4. Repartición por persona
- 5. Pruebas de carga: quién corre qué y cómo se mantiene la comparabilidad
- 6. Cronograma de cuatro semanas y dependencias
- 7. Reglas de trabajo y definición de terminado
- 8. Actualización del 12 de octubre: lo que ya está desplegado y lo que necesitan Samara y Mateo
- 9. Cómo trabajamos en paralelo: prioridades y dependencias de esta semana
- 10. Repartición del frontend

1. Estado de partida

1.1 Correcciones de la Entrega 2 (nota 76,67/100)

Registro completo con scripts y evidencia en `docs/entrega2/correcciones-retroalimentacion.md`. Todas quedaron cerradas entre el 9 y el 10 de octubre de 2026.

Id	Observación	Qué se hizo	Estado y evidencia
C1	Login del Web Server real devolvía 500	Verificado `REDIS_ADDR` del Web Server hacia la IP interna del Worker tras el cambio a mooc-e2-vpc	Cerrada: registro, verificación y login 200 (0,45 s)
C2	Cloud SQL con IP pública y redes autorizadas	Private Service Access (rango mooc-e2-private-services), IP privada, `DATABASE_URL` de ambas VM a la IP privada, IP pública y redes autorizadas eliminadas	Cerrada: solo 10.235.0.3, ipv4Enabled false, sslMode ENCRYPTED_ONLY, migraciones en versión 5
C3	Worker con IP externa	Cloud NAT mooc-e2-nat (router mooc-e2-nat-router), Private Google Access, IP externa retirada	Cerrada: Worker solo con 10.20.0.2; salida a Cloud Storage, Artifact Registry y firmas de ClamAV verificada
C4	Flujo multimedia probado contra una API temporal dentro del Worker	Escenario 2 repetido contra el Web Server real por HTTPS (`k6/correr-escenario2-cloud.ps1`)	Cerrada: con e2-small el Worker colapsa (Redis en la misma VM, login 500); ajuste mínimo a e2-medium y concurrencia 2: 278/278 y 300/300 checks, 0 % errores HTTP
C5	Archivos trasladados sin comprobar integridad; release sin resolver	Verificación de existencia, tamaño, SHA-256, HLS y PDF de cada media_asset (`07-verificar-integridad-objetos.ps1`)	Cerrada: 73 recursos ready íntegros, 10/10 SHA-256 coinciden, 3 filas en failed. Reporte en docs/entrega2/integridad-objetos.md
C6	Hallazgo propio: imagen de la API anterior a cookies y CSRF	Imagen reconstruida y fijada por commit (`mooc-e2-api:298301b`)	Cerrada: login emite mooc_session (Secure, HttpOnly, SameSite=Lax) y mooc_csrf

1.2 Estado actual de GCP (región us-central1, zona us-central1-a)

- **Red:** VPC `mooc-e2-vpc`, subred `mooc-e2-subnet` 10.20.0.0/24, Cloud NAT, Private Google Access, SSH solo por IAP (35.235.240.0/20).
- **Web Server** `mooc-e2-web`: Docker Compose con API (`mooc-e2-api:298301b`) y Caddy con HTTPS sobre sslip.io.
- **Worker Server** `mooc-e2-worker`: e2-medium (subió desde e2-small en C4), `WORKER_CONCURRENCY=2`; aloja worker, Redis y ClamAV.
- **Cloud SQL** `mooc-postgres`: PostgreSQL solo por IP privada; las migraciones ya no corren desde un portátil, sino desde una VM de la VPC.
- **Cloud Storage** con claves HMAC (S3 compatible) y **Artifact Registry** con imágenes etiquetadas por commit.
- **Cuidado con el costo:** el Worker sigue en e2-medium y Cloud SQL sigue encendida. Apagarlas cuando no se esté probando.

1.3 Lo que heredamos y debe atenderse en la Entrega 3

Estos puntos salieron de las correcciones. Unos se resuelven con el nuevo diseño; los que queden abiertos van al inventario de deuda técnica.

Hallazgo	Por qué importa en la E3	Dónde se atiende
Redis y ClamAV viven en la VM del Worker (causa del colapso de C4)	La E3 prohíbe que la cola dependa de una máquina que el autoscaling pueda retirar	VM de servicios (Mateo)
El `/health` de la API responde ok sin revisar Postgres ni Redis	El balanceador debe dejar de enviar tráfico a una instancia que no puede atender	`/health/ready` (Daniela)
La API no hace apagado ordenado; asynq usa el ShutdownTimeout por defecto (8 s)	Al reducir un grupo se pierden solicitudes en curso o se cortan transcodificaciones	Daniela (API), Mateo (workers)
Mailpit publicado a través de Caddy en el Web Server	Superficie expuesta innecesaria; no debe pasar a la plantilla del MIG	Fuera de la plantilla; registrar en deuda (Samara)
3 media_assets en failed y objetos sin fila asociada (integridad-objetos.md)	Afectan la reconciliación de trabajos de 3B	Limpieza o registro (Camila con Mateo)
Hallazgo de reprocesamiento con Idempotency-Key del informe de la E2	La E3 exige reintentos sin efectos duplicados	Prueba de integridad (Camila)
Push y tag `entrega-2` en GitHub	La línea base debe poder reconstruirse desde un commit identificable	Confirmar en GitHub (Daniela)

2. Qué exige la Entrega 3

Resumen del enunciado (13 páginas). Lo que no cumple el componente está marcado en negrilla porque son los errores que hacen perder más puntos.

- **Dos grupos independientes** (API y workers), cada uno de 1 a 3 instancias, con escalamiento automático. **No vale ampliar a mano ni escalar solo la API.** Subir procesos dentro de una máquina no cuenta como escalamiento horizontal.
- **Balanceador externo con HTTPS** y health checks (endpoint, protocolo, intervalo, timeout, umbrales). Demostrar reparto entre dos instancias sanas y exclusión de una que falla.
- **Arranque autónomo**: plantilla por grupo, sin acceso manual, reinicios sin duplicar procesos, migraciones controladas (no cada réplica), tiempo de arranque medido y criterio de listo.
- **Cola compartida fuera de ambos grupos**; sesiones válidas entre réplicas; estado de negocio solo en Cloud SQL y Cloud Storage; retiro seguro de workers con trabajos activos.
- **Frontend mínimo funcional** desde el navegador, sobre el entorno cloud: recorrido conectado profesor (crea, carga, quiz, publica) y estudiante (encuentra, se inscribe, consume, progreso, quiz). **No vale reemplazar pasos con llamadas manuales a la API.** Matriz de flujos, rutas y evidencias.
- **Pruebas**: repetir Escenarios 1 y 2 con la misma herramienta y parámetros, y el Escenario 3 en dos variantes (3A API y 3B workers) con ciclo completo de subida y bajada. Tabla comparativa E2 contra E3.
- **Calidad**: SonarQube en GitHub Actions, duplicación de 20 % o menos, mantenibilidad C o mejor, deuda total registrada; `docs/entrega3/deuda-tecnica.md` con inventario priorizado.
- **Documentación y cierre**: `docs/entrega3/arquitectura.md`, `capacity-planning/pruebas_de_carga_entrega3.md`, costos con 1 y 3 instancias por grupo, tag `entrega-3` y video. Todos deben poder explicar todo.
- No se exige: multizona, failover de base de datos, CDN, Kubernetes, PaaS/serverless ni despliegue continuo.

3. Propuesta de despliegue eficiente

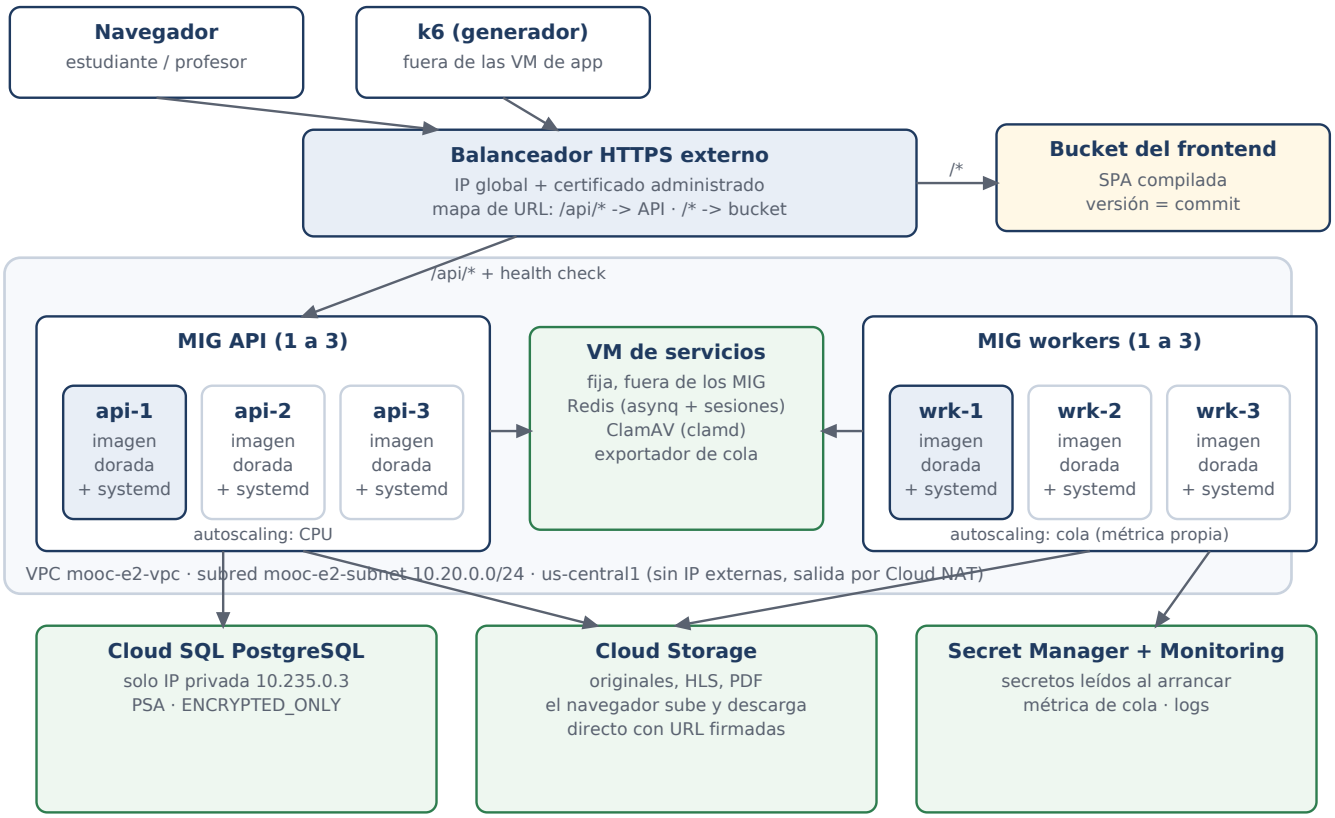
3.1 El problema

En la Entrega 2 cada VM se preparaba una vez y se desplegaba con Docker Compose por SSH. Con grupos que crean y destruyen máquinas solas, ese modelo deja de servir: cada máquina nueva tendría que instalar Docker, autenticarse, descargar la imagen y levantar Compose antes de atender tráfico. Eso tarda minutos, depende de repositorios externos y puede fallar a mitad de un pico de carga. Docker en sí no es el cuello de botella; lo lento es **instalar y descargar en cada arranque**. El enunciado además pide conservar los contenedores Docker, así que la mejora consiste en preparar todo antes.

Opción	Arranque típico	Ventajas	Problemas	Decisión
Script de arranque que instala Docker y descarga la imagen en cada VM	Varios minutos	Simple de escribir	Lento, depende de apt y del registro en cada arranque, más puntos de falla	Descartada
Imagen de máquina por rol con Docker y el contenedor ya descargado + systemd + script corto	Del orden de un minuto (a medir)	Rápido, reproducible, misma versión en todo el grupo	Hay que construir una imagen nueva por versión (script)	**Elegida**
Binario Go nativo con systemd, sin Docker	El más rápido	Sin descarga de imagen	Rompe la continuidad con contenedores; ffmpeg, LibreOffice y ClamAV a mano	Descartada
Container-Optimized OS con declaración de contenedor	Rápido	Poco mantenimiento	Google marcó como obsoleto ese mecanismo de declaración	Descartada

Los tiempos son referencias para orientar la decisión; el tiempo real de arranque de cada grupo se mide y se reporta (lo exige el enunciado).

3.2 Arquitectura objetivo



Componentes fijos (fuera de los límites 1 a 3): balanceador, bucket del frontend, VM de servicios, Cloud SQL, Cloud Storage. Componentes elásticos: MIG de API y MIG de workers. Ambos grupos usan Cloud SQL (IP privada) y Cloud Storage; las flechas muestran solo los caminos principales. Las cargas de archivos siguen yendo directo a Cloud Storage con URL firmadas, como exige el enunciado.

3.3 Cómo arranca una instancia nueva, paso a paso

El mismo patrón sirve para API y workers; cambia la imagen y el contenedor.

- **Imagen de máquina** (familias `mooc-api` y `mooc-worker`): Debian o Ubuntu LTS, Docker Engine, el contenedor ya descargado con la etiqueta del commit y una unidad systemd `mooc.service`. Se construye con un script desde una VM temporal y se versiona con el commit en el nombre (ej. `mooc-api-298301b`).
- **Plantilla de instancia**: tipo de máquina, disco, imagen, subred sin IP externa, etiqueta de red, cuenta de servicio del rol y metadatos no sensibles (`rol`, `version`, `redis-addr`, nombre de los secretos).
- **Script de arranque** (corto e idempotente): lee los metadatos, pide los secretos a Secret Manager con la identidad de la VM, escribe `/run/mooc/env` con permisos 600 (en memoria, no en disco persistente) y hace `systemctl restart mooc`.
- **Unidad systemd**: `docker rm -f mooc` y luego `docker run --name mooc --env-file /run/mooc/env ...`. El nombre fijo evita duplicar contenedores al reiniciar; `TimeoutStopSec` da tiempo al apagado ordenado.

- **Listo** significa: `/health/ready` responde 200 porque la API alcanza Postgres y Redis (API), o el worker se registró en asynq y consume de `media` (workers). El tiempo se mide desde la creación de la VM hasta ese momento.
- **Nueva versión**: construir el contenedor, construir la imagen de máquina, crear plantilla nueva y aplicar una actualización progresiva del MIG. Así todas las instancias de una corrida tienen la misma versión.

3.4 Frontend sin máquinas virtuales

La duda de cuántas máquinas poner en el front se resuelve sin máquinas: una SPA compilada (HTML, JS y CSS) no necesita servidor de aplicación. Propuesta:

- **Tecnología:** React + Vite + TypeScript, `react-router` para las rutas y `hls.js` para el video. El enunciado general deja la tecnología a elección del equipo.
- **Ubicación:** bucket de Cloud Storage conectado al balanceador como backend bucket. El mapa de URL envía `/api/*` al servicio de backend de la API y todo lo demás al bucket.
- **Mismo dominio:** navegador, frontend y API comparten el origen HTTPS, así que las cookies `mooc_session` (HttpOnly, SameSite=Lax) y el encabezado `X-CSRF-Token` funcionan sin configurar CORS. Se documenta igual, como pide el enunciado.
- **Versión identificable:** cada publicación sube los archivos con un `version.json` que tiene commit y fecha, visible en el pie de la app.
- **Rutas de la SPA:** configurar la página de error del bucket a `index.html` para que las rutas internas recarguen bien.
- **Costo:** unos centavos de almacenamiento; no cuenta dentro de los límites 1 a 3.

3.5 Políticas de escalamiento (valores iniciales a calibrar)

Los números son el punto de partida; se ajustan con las primeras corridas y la justificación final se escribe con lo observado.

Parámetro	MIG de API	MIG de workers
Tipo de máquina	e2-small, igual que el Web Server de la E2, para comparar	e2-medium, igual que el ajuste de C4; WORKER_CONCURRENCY=2 por instancia
Señal	Utilización de CPU del grupo	Métrica propia en Cloud Monitoring: trabajos pendientes + activos de la cola media (un exportador en la VM de servicios la publica cada 30 s)
Objetivo	60 % de CPU	Alrededor de 4 trabajos por instancia (2 en ejecución + 2 en espera) con asignación por instancia
Inicialización	Medida; arrancar con unos 120 s	Medida; arrancar con unos 120 s
Reducción	Controles de reducción: retirar como máximo 1 instancia por cada 10 min	Igual, y la señal incluye los activos para no retirar workers con una cola vacía pero ocupada
Health check	Balanceador: GET /health/ready, cada 5 s, timeout 3 s, 2 aciertos o 2 fallos. Autorreparación: misma ruta, umbral más tolerante y retraso inicial de unos 180 s	Autorreparación: endpoint mínimo del worker (ej. :8081/health) que verifica Redis y el servidor asynq
Límites	Mínimo 1, máximo 3; al llegar a 3 la latencia sube y se documenta	Mínimo 1, máximo 3; al llegar a 3 la cola crece y se documenta la espera

Por qué la cola y no la CPU en workers: ffmpeg satura la CPU con un solo trabajo, así que la CPU no distingue entre 1 y 20 trabajos en espera. El enunciado pide expresamente una señal de

profundidad o antigüedad de cola.

3.6 Retiro seguro de workers

- Cuando el MIG retira una VM, el sistema operativo detiene `mooc.service`; Docker envía SIGTERM al worker.
- asynq deja de tomar trabajos nuevos al recibir SIGTERM y espera a los activos durante `ShutdownTimeout`. Hoy usa 8 s por defecto: se sube a un valor mayor que la duración de un trabajo (por ejemplo 300 s) y se alinea con `TimeoutStopSec`.
- Para que la VM espere, se habilita el apagado ordenado de Compute Engine en la plantilla si el proyecto lo permite; si no, se usa un shutdown script, que tiene un margen más corto.
- Si el trabajo no alcanza a terminar, asynq lo devuelve a la cola cuando vence su arrendamiento y otro worker lo toma. El `idempotency_key` y el chequeo de estado en Postgres evitan derivados duplicados. Esto se demuestra en 3B.

3.7 Migraciones, secretos e identidades

- **Migraciones:** nunca en el arranque de las réplicas. Un script corre `migrate/migrate` una sola vez desde la VM de servicios, antes de la actualización progresiva. golang-migrate ya bloquea la base mientras migra.
- **Secretos:** `DATABASE_URL`, claves HMAC y contraseña SMTP pasan a Secret Manager. Ni las imágenes, ni las plantillas, ni los scripts versionados los contienen. Se entrega `*.env.example` sin valores.
- **Identidades:** una cuenta de servicio por rol (`sa-mooc-api`, `sa-mooc-worker`) con permiso de leer solo sus secretos, leer de Artifact Registry y escribir logs y métricas. Se documenta aparte la autenticación de la app ante PostgreSQL (usuario y clave en la cadena de conexión).
- **Firewall:** API solo desde los rangos del balanceador y de los health checks de Google (130.211.0.0/22 y 35.191.0.0/16) y SSH por IAP; workers sin entrada; Redis (6379) y ClamAV (3310) solo desde las etiquetas de API y workers.

3.8 Cambios mínimos de código

Cambio	Para qué	Quién
`/health/ready` que verifica Postgres y Redis con timeout corto (sin datos sensibles)	Health check real del balanceador	Daniela
Apagado ordenado de la API: al recibir SIGTERM, responder 503 en ready, esperar el drenaje y llamar a `e.Shutdown`	No cortar solicitudes en curso al reducir	Daniela
Encabezado `X-Instance-Id` (nombre de la VM) en las respuestas	Probar el reparto entre réplicas en 3A y en el video	Daniela
`ShutdownTimeout` configurable, endpoint de salud y dirección de ClamAV por variable en el worker	Retiro seguro y ClamAV fuera del worker	Mateo
Exportador de la cola (asynq Inspector a Cloud Monitoring)	Señal de autoscaling de workers	Mateo
Pruebas automáticas de las correcciones (health, apagado, idempotencia)	El enunciado pide mantener las pruebas en CI	Cada responsable

Ruta propuesta para los scripts: `deploy/gcp/e3/`, numerados en el orden de reconstrucción, con un `99-retirar-recursos.ps1` para bajar costos y reconstruir antes de la sustentación.

4. Repartición por persona

Cada quien sigue con lo que conoce desde la Entrega 1 (su módulo del backend) y la Entrega 2 (su parte de infraestructura). El cambio grande es que Samara y Camila corren las pruebas de carga, con acompañamiento de quien las corrió antes. Todas las personas tocan frontend, porque cada pantalla consume el módulo que esa persona construyó.

Persona	Hizo antes	Frente principal en la E3
Daniela Solarte	E1: cursos, autoría y catálogo. E2: empaquetado, imágenes, scripts de despliegue, Escenario 1, documentación y correcciones	MIG de API, imágenes de máquina, secretos, migraciones, pantallas de profesor y catálogo, arquitectura.md
Mateo	E1: multimedia (carga, ClamAV, HLS). E2: ubicación de Redis, Cloud Storage, URL firmadas, workers, observabilidad, costos, Escenario 2	VM de servicios, MIG de workers, métrica de cola, retiro seguro, pantallas multimedia, costos
Samara Martínez	E1: identidad, sesiones y administración. E2: CI y seguridad de cookies y CSRF	Balanceador HTTPS y firewall, base del frontend y login, SonarQube, deuda técnica, **Escenarios 1 y 3A**
Camila Fajardo	E1: quizzes, intentos, progreso e insignias. E2: Cloud SQL	Pantallas de estudiante y quiz, integridad entre réplicas, **Escenarios 2 y 3B**, matriz de verificación del frontend

En los commits de la E2 la parte de Redis, workers, Escenario 2 y costos aparece a nombre de Andrés Jurado; en este documento corresponde a Mateo.

Daniela Solarte · API elástica, despliegue y arquitectura

Sigue con el empaquetado y los scripts de despliegue que hizo en la E2 y con su módulo de cursos y catálogo.

Infraestructura

- Script que construye la imagen de máquina por rol (lo usa también Mateo para workers) y la versiona con el commit.
- Plantilla y MIG de API: script de arranque, unidad systemd, autoscaling por CPU, autorreparación, actualización progresiva.
- Secret Manager y cuentas de servicio por rol con permisos mínimos.
- Script de migraciones controladas desde la VM de servicios.
- Confirmar el push y el tag entrega-2; apagar el Web Server y el Worker de la E2 cuando el nuevo entorno funcione (después de conservar evidencias).

Backend

- /health/ready, apagado ordenado de la API y X-Instance-Id.

Frontend

- Profesor: mis cursos, crear y editar curso con módulos y unidades, agregar recursos, publicar y ver el resultado de la publicación (con los errores de reglas de negocio).
- Estudiante: catálogo, detalle de curso publicado e inscripción.

Pruebas

- Medir y registrar el tiempo de arranque de la API.
- Entregar a Samara el traspaso del Escenario 1 (k6/escenario1.js, runner, parámetros, resultados de la E2).

Documentación

- docs/entrega3/arquitectura.md (componentes, despliegue, decisiones, operación, reconstrucción, limitaciones) y README.
- Coordinar el tag entrega-3 y el guion del video.

Terminado cuando: el MIG de API crea una instancia sola que pasa el health check sin intervención, la API responde por el balanceador con la versión fijada, y un profesor crea y publica un curso desde el navegador.

Mateo · Workers elásticos, cola compartida y multimedia

Sigue con lo que decidió en la E2 (ubicación de Redis, configuración de workers, costos) y con su módulo multimedia.

Infraestructura

- VM de servicios fija con Redis (persistencia AOF) y ClamAV; mover la cola y las sesiones a esa VM sin perder trabajos.
- Imagen de máquina, plantilla y MIG de workers con autoscaling por la métrica de cola.
- Exportador de la cola a Cloud Monitoring y tablero con instancias por grupo, cola y CPU (sirve para las pruebas y el video).
- Apagado ordenado de las VM de worker (configuración de la plantilla y de systemd).

Backend

- ShutdownTimeout configurable, endpoint de salud del worker, ClamAV por red.

Frontend

- Profesor: carga de archivos (URL firmadas y multipart), estado de procesamiento. Un archivo en proceso no se muestra como disponible.
- Estudiante: reproductor HLS y visor de PDF y documentos.

Pruebas

- Medir el tiempo de arranque de un worker.
- Entregar a Camila el traspaso del Escenario 2 (orquestador correr-escenario2-cloud.ps1, perfiles de video, monitores, resultados de la E2) y acompañar su primera corrida.

Documentación

- Costos: ambos grupos con 1 y con 3 instancias, frontend, balanceador, VM de servicios, base de datos, objetos, red y monitoreo; alerta de presupuesto.
- Sección de cola, workers y retiro seguro en arquitectura.md.

Terminado cuando: con la cola llena, el MIG crea un worker nuevo que empieza a procesar solo; al vaciarse, lo retira sin perder trabajos; y un estudiante ve un video procesado desde el navegador.

Samara Martínez · Entrada segura, frontend base, calidad y Escenarios 1 y 3A

Sigue con identidad, sesiones y CI, que fueron su parte en la E1 y la E2, y esta vez corre pruebas de carga.

Infraestructura

- Balanceador HTTPS externo: IP global, certificado administrado (dominio sslip.io con la IP del balanceador), mapa de URL /api/* a la API y /* al bucket, health check y drenaje de conexiones.
- Reglas de firewall de la E3 (balanceador, health checks, IAP, servicios internos).
- Bucket del frontend y script de publicación con version.json.

Backend

- Revisar que las sesiones y el CSRF sigan funcionando detrás del balanceador (cookies Secure, encabezados reenviados).

Frontend

- Base del proyecto (React + Vite + TypeScript), cliente HTTP con cookies y X-CSRF-Token, login y logout, rutas protegidas por rol, componentes de cargando, vacío, error y éxito.
- Verificar en la interfaz que un estudiante no accede a acciones de profesor.

Pruebas

- **Escenario 1** sobre el balanceador con los mismos parámetros de la E2.
- **Escenario 3A**: ciclo de subida y bajada del MIG de API con línea de tiempo (carga, CPU, decisión del autoscaler, instancia creada, incorporada al balanceador, retirada) y prueba de exclusión de una instancia que falla el health check.

Documentación

- SonarQube en GitHub Actions: primer análisis temprano (diagnóstico inicial) y reporte del commit final.
- docs/entrega3/deuda-tecnica.md: arma el documento; cada persona aporta los hallazgos de su área.

Terminado cuando: el frontend se abre por HTTPS desde el balanceador con login funcional, el pipeline publica el análisis de SonarQube, y 3A muestra al menos una subida y una bajada automáticas con dos instancias atendiendo.

Camila Fajardo · Recorrido del estudiante, integridad y Escenarios 2 y 3B

Sigue con quizzes, intentos y progreso, su módulo de la E1, y con Cloud SQL de la E2; esta vez corre las pruebas multimedia y de workers.

Infraestructura

- Revisar conexiones y carga de Cloud SQL con varias réplicas (máximo de conexiones y tamaño del pool por instancia).

Backend

- Corregir lo que salga de las pruebas de integridad en su módulo (doble envío del quiz, progreso).

Frontend

- Estudiante: mis cursos, navegación de módulos, unidades y recursos, registro de progreso, presentar el quiz y ver el resultado.
- Profesor: crear o editar un quiz básico.

Pruebas

- Pruebas de integridad entre réplicas: sesión que alterna entre instancias, lecturas y escrituras consistentes, intento enviado dos veces sin doble calificación, archivos accesibles desde cualquier instancia, datos conservados al retirar una API.
- **Escenario 2** sobre el entorno elástico con los perfiles de la corrección C4.
- **Escenario 3B**: ciclo de subida y bajada del MIG de workers con línea de tiempo, drenaje de la cola, retiro controlado de un worker con un trabajo activo y reconciliación final de trabajos.

Documentación

- capacity-planning/pruebas_de_carga_entrega3.md: arma el informe con Samara y la tabla comparativa E2 contra E3.
- Matriz de verificación del frontend (flujos, rutas, evidencias) en docs/entrega3.

Terminado cuando: el recorrido conectado profesor y estudiante corre completo en el navegador sobre la nube, y 3B muestra un worker nuevo procesando y un retiro sin trabajos perdidos ni duplicados.

5. Pruebas de carga

Escenario	Corre	Acompaña	Base de comparación de la E2
1. Actividad académica	Samara	Daniela	Resultados e1-gcp de la E2 (k6/escenario1.js)
2. Carga y procesamiento multimedia	Camila	Mateo	Repetición de C4 contra el Web Server real (e2-medium, concurrencia 2)
3A. Autoscaling de la API	Samara	Daniela	No aplica (escenario nuevo)
3B. Autoscaling de workers y retiro seguro	Camila	Mateo	No aplica (escenario nuevo)

Reglas para que la comparación sea válida

- Misma herramienta y versión (k6 v2.3.0 en Docker Desktop), mismos datos, mezcla, pausas, duración, calentamiento y perfiles de video que en la E2. Toda diferencia se declara.
- La concurrencia por worker se mantiene en 2 sobre e2-medium; lo que cambia es el número de workers, que es el efecto que se estudia.
- El generador corre fuera de las VM de la aplicación. Registrar sus recursos y comprobar que no limita (en la E2 una suspensión del portátil causó 4 fallos en una corrida). Si en 3A el portátil no alcanza a subir la CPU, usar una VM generadora y declararlo.
- Las solicitudes a la API entran por el balanceador; las transferencias de archivos van directo a Cloud Storage.
- k6 descarta las cookies tras el login y usa Bearer (si no, el CSRF responde 403); esto ya está resuelto en los scripts.
- Antes de correr: criterios de éxito, degradación y parada por escrito. Repetir las corridas comparadas para informar variación. No promediar percentiles entre corridas.
- Correr todo en ventanas acordadas y apagar después; Cloud SQL y los grupos cobran por hora.

Traspaso

En la semana 1, Daniela y Mateo hacen una sesión corta con Samara y Camila: cómo se lanza cada script, qué se monitorea, dónde quedan los resultados y qué falló en la E2 (colapso del Worker e2-small, timeout del setup, cookies y CSRF). Samara y Camila hacen una corrida de práctica contra el entorno de la E2 antes de que existan los grupos, para no aprender la herramienta durante las corridas evaluadas.

6. Cronograma de cuatro semanas

Semana	Daniela	Mateo	Samara	Camila
1 · Base	Imagen de máquina de API, secretos y cuentas de servicio, /health/ready	VM de servicios (Redis y ClamAV), migrar la cola, imagen de worker	Primer análisis de SonarQube, base del frontend y login, borrador del balanceador	Corrida de práctica de los escenarios, pantallas de estudiante (mis cursos y navegación)
2 · Grupos	MIG de API con autoscaling, migraciones controladas, pantallas de profesor	MIG de workers, exportador de cola, retiro seguro, pantallas multimedia	Balanceador con certificado y bucket, firewall, rutas por rol	Quiz (estudiante y profesor), progreso, pruebas de integridad entre réplicas
3 · Pruebas	Apoyo en 3A, publicación del curso, arquitectura.md	Apoyo en 3B, tablero de monitoreo, costos	Escenarios 1 y 3A con repeticiones	Escenarios 2 y 3B con repeticiones y retiro controlado
4 · Cierre	Recorrido conectado en la nube, README, tag entrega-3, video	Costos finales, retiro de recursos y guía de reconstrucción	Análisis final de SonarQube, deuda-tecnica.md	Informe de capacidad y tabla comparativa, matriz del frontend

Dependencias que marcan el ritmo

- La VM de servicios (Mateo) va primero: sin cola compartida no hay MIG de API ni de workers.
- El MIG de API (Daniela) y el balanceador (Samara) se integran a mitad de la semana 2; hasta entonces el frontend se desarrolla contra Docker Compose local.
- Los escenarios evaluados empiezan cuando ambos grupos y el balanceador estén estables (semana 3). El análisis de SonarQube corre desde la semana 1 para tener el diagnóstico inicial.
- El recorrido conectado del frontend en la nube es el último bloqueo antes del video.

7. Reglas de trabajo y definición de terminado

- Ramas `entrega3/<nombre>` y pull request a `main` con el CI en verde. Nada se despliega desde una rama sin fusionar.
- Ningún secreto en código, imágenes, plantillas, scripts, capturas ni video. Revisar con el secret guard del CI antes del tag.
- Todo cambio en GCP queda en un script de `deploy/gcp/e3/`; nada hecho a mano en la consola sin registrarlo.
- Cada evidencia con fecha UTC, commit y nombre de corrida; resultados originales en el repositorio.
- Todos deben poder explicar las cuatro partes en la sustentación: cada quien prepara una explicación de cinco minutos de su frente para los demás en la semana 4.
- Apagar Cloud SQL, los grupos y la VM de servicios al final de cada sesión de pruebas.

Entregables finales y responsable

Entregable	Responsable	Apoyo
Plataforma desplegada (grupos, balanceador, cola, persistencia)	Daniela y Mateo	Samara
Frontend mínimo y recorrido conectado	Samara (base)	Todos
docs/entrega3/arquitectura.md	Daniela	Mateo, Samara
Matriz de verificación del frontend (docs/entrega3)	Camila	Todos
capacity-planning/pruebas_de_carga_entrega3.md	Camila y Samara	Daniela, Mateo
docs/entrega3/deuda-tecnica.md y reporte de SonarQube	Samara	Todos
Costos y retiro y reconstrucción de recursos	Mateo	Daniela
Tag entrega-3, README y video	Daniela	Todos

8. Actualización del 12 de octubre de 2026

Daniela ya desplegó el grupo de la API con el mecanismo de la sección 3. Esta sección deja lo que Samara y Mateo necesitan para conectar sus partes sin repetir trabajo. Todo se creó con los scripts de `deploy/gcp/e3/` (ver su README) y los registros quedan en `deploy/gcp/logs/e3-*.log`.

8.1 Lo que ya existe en GCP

Recurso	Nombre	Detalle
Cuentas de servicio	`sa-mooc-api`, `sa-mooc-worker`	Escribir logs y métricas, leer el repositorio `mooc-e2` de Artifact Registry y leer los 3 secretos
Secretos (Secret Manager)	`mooc-database-url`, `mooc-s3-access-key`, `mooc-s3-secret-key`	Copiados del Web Server de la E2; solo los leen las dos cuentas de servicio
Contenedor de la API	`mooc-e2-api:ca471a0`	Incluye `/health/ready`, apagado ordenado y `X-Instance-Id`
Imagen de máquina	`mooc-api-ca471a0` (familia `mooc-api`)	Docker + contenedor ya descargado + `mooc.service`; se construyó en 5 min
Plantilla	`mooc-api-ca471a0-10122103`	e2-small, sin IP externa, etiqueta `mooc-api`, cuenta `sa-mooc-api`
Health checks	`mooc-api-hc-listo` y `mooc-api-hc-vida`	Listo: /health/ready cada 5 s, timeout 3 s, 2/2. Vida: /health cada 10 s, timeout 5 s, 2/3
Grupo de la API	`mooc-api-mig` (us-central1-a)	1 a 3 instancias, 60 % de CPU, 120 s de inicialización, reducción de 1 instancia cada 10 min, autorreparación con retraso de 180 s
Firewall	`mooc-e3-allow-hc-api`, `mooc-e3-allow-iap-ssh`, `mooc-e3-redis-temporal`	Rangos de Google a la API:8080; SSH por IAP a mooc-api, mooc-worker y mooc-servicios; Redis temporal (ver 8.3)

Prueba del 12 de octubre: la instancia `mooc-api-k580` respondió `200` en `/health/ready` con Postgres y Redis en ok y `X-Instance-Id: mooc-api-k580`. Tiempo de arranque: **28 s** desde la creación de la VM hasta el contenedor de la API en marcha. Sin acceso manual.

8.2 Para Samara: balanceador HTTPS sobre el grupo de la API

El grupo ya está listo para conectarse. No hay que crear grupo, plantilla, health check ni regla de firewall de la API: solo el balanceador.

- **Servicio de backend:** protocolo HTTP, puerto con nombre `http` (ya configurado en el grupo, apunta al 8080), backend `mooc-api-mig` en la zona `us-central1-a`.
- **Health check del backend:** `mooc-api-hc-listo`. No usar `mooc-api-hc-vida`, que es solo para la autorreparación del grupo.
- **Drenaje de conexiones:** 30 s. La API, al recibir la señal de apagado, pasa `/health/ready` a 503, espera 10 s y luego hasta 20 s más por las solicitudes en curso.
- **Firewall:** `mooc-e3-allow-hc-api` ya permite `130.211.0.0/22` y `35.191.0.0/16` hacia `tcp:8080` en la etiqueta `mooc-api`.
- **Mapa de URL:** `/api/*` al servicio de backend de la API y el resto al bucket del frontend (sección 3.4).
- **Evidencia de reparto para 3A:** cada respuesta trae `X-Instance-Id` con el nombre de la VM; basta contar ese encabezado en una ráfaga de solicitudes por el balanceador.

- **Cookies:** la API emite `mooc_session` (Secure, HttpOnly, SameSite=Lax) y `mooc_csrf`; con el balanceador terminando HTTPS y el frontend en el mismo dominio no hace falta CORS.
- **Encender y apagar el grupo:** `.\deploy\gcp\e3\03-grupo-api.ps1 -Fase Encender` / `-Fase Apagar`; estado con `-Fase Estado`. El grupo no se escala a mano: eso no cuenta como autoscaling.

8.3 Para Mateo: grupo de workers y VM de servicios

El mecanismo de arranque es el mismo para los dos roles; el worker reutiliza lo que ya existe.

- **Identidad y secretos:** `sa-mooc-worker` ya existe y ya puede leer `mooc-database-url`, `mooc-s3-access-key` y `mooc-s3-secret-key`, escribir métricas (para el exportador de la cola) y leer de Artifact Registry.
- **Imagen de máquina:** `.\deploy\gcp\e3\02-imagen-maquina.ps1 -Rol worker` con Docker Desktop encendido y el código sin cambios pendientes. Deja `mooc-worker-<commit>` en la familia `mooc-worker`.
- **Plantilla:** partir de `03-grupo-api.ps1` (fase Plantilla) y de `config/api.config`. Metadatos que lee `mooc-preparar.sh` en cada arranque:

Metadato	Valor para el worker
`mooc-imagen`	`us-central1-docker.pkg.dev/proyecto1-entrega2-desarrollo/mooc-e2/mooc-e2-worker:<commit>`
`mooc-config`	Líneas CLAVE=valor sin secretos: `REDIS_ADDR`, `S3_ENDPOINT`, `S3_BUCKET`, `S3_USE_SSL`, `S3_REGION`, `WORKER_WORKDIR`, `WORKER_CONCURRENCY=2` (igual que la corrección C4, para comparar), `DB_MAX_CONNS`
`mooc-secretos`	`DATABASE_URL=mooc-database-url`, `S3_ACCESS_KEY=mooc-s3-access-key`, `S3_SECRET_KEY=mooc-s3-secret-key`
`mooc-detener-s`	Mayor que el `ShutdownTimeout` de asynq (por ejemplo 330 si el timeout es 300), para que `docker stop` no corte una transcodificación
`mooc-docker-args`	`--add-host=clamav:<IP de la VM de servicios>`: la imagen del worker busca ClamAV con el nombre `clamav` (está fijo en su clamd.conf)

- **Red:** etiqueta `mooc-worker`, sin IP externa; la regla `mooc-e3-allow-iap-ssh` ya le permite SSH por IAP. Los workers no necesitan ninguna regla de entrada.
- **Redis temporal:** mientras no exista la VM de servicios, la API usa el Redis del Worker de la E2 (`10.20.0.2:6379`) por la regla `mooc-e3-redis-temporal`. No apagar `mooc-e2-worker` mientras el grupo de la API esté encendido.
- **Cuando exista la VM de servicios:** crearle la etiqueta `mooc-servicios` y una regla que permita `tcp:6379` y `tcp:3310` solo desde `mooc-api` y `mooc-worker`; pasarle la IP a Daniela, que crea una plantilla nueva con `-RedisAddr <IP>:6379`, aplica `-Fase Actualizar` y borra `mooc-e3-redis-temporal`.
- **Migraciones:** se mueven a la VM de servicios (Daniela). Hasta entonces se ejecutan una sola vez con `08-publicar-imagen-api.ps1 -AplicarMigraciones` desde el Web Server.

9. Cómo trabajamos en paralelo esta semana

Nadie puede terminar su parte completa antes de que empiece la siguiente persona: las piezas dependen unas de otras. La regla es **primero lo que desbloquea a los demás**. Cada quien entrega primero su pieza de la que dependen otros y, mientras espera las de los demás, avanza en lo que no depende de nadie.

9.1 Lo que desbloquea a otros (en este orden)

#	Quién	Qué	A quién desbloquea	Fecha meta
1	Daniela	Llevar a `main` en GitHub las correcciones de la E2 (hoy `main` está 36 commits atrás), crear el tag `entrega-2` y el pull request de `entrega3/daniela`	A todos: las ramas nuevas deben salir de un `main` al día	Lunes 12 oct
2	Samara	Base del frontend (sección 10.1): proyecto, cliente HTTP con cookies y CSRF, login, rutas por rol y componentes de estados	A todos en el frontend	Miércoles 14 oct
3	Daniela	Endpoint nuevo para que el profesor liste **sus** cursos (hoy la API no lo tiene: el catálogo solo muestra publicados)	Pantalla Mis cursos del profesor	Miércoles 14 oct
4	Mateo	**VM de servicios** con Redis (cola y sesiones) y ClamAV, fuera de los grupos; regla de firewall solo desde `mooc-api` y `mooc-worker`	A Daniela (migraciones y Redis definitivo de la API) y a su propio grupo de workers	Jueves 15 oct
5	Samara	Balanceador HTTPS sobre `mooc-api-mig` y bucket del frontend (sección 8.2)	Frontend en la nube y Escenarios 1 y 3A	Viernes 16 oct
6	Mateo	Imagen y grupo de workers con autoscaling por la cola	Escenarios 2 y 3B (Camila)	Lunes 19 oct
7	Samara	Cuentas de prueba (administrador, profesor, estudiante) con un script, en local y en la nube	Pruebas del frontend de todos	Miércoles 14 oct

9.2 Lo que cada quien puede avanzar mientras espera

- **Todos:** levantar el backend en local con `docker compose up -d --build` y desarrollar el frontend contra `localhost:8080`. No hace falta la nube para construir pantallas; la nube se usa para probar y para la evidencia.
- **Camila:** corrida de práctica de los escenarios con el traspaso de Mateo, plan de las pruebas de integridad entre réplicas (ya se pueden hacer con el grupo de la API escalado a 2) y componentes del quiz aislados.
- **Samara:** primer análisis de SonarQube en GitHub Actions (es el diagnóstico inicial; no depende de nada).
- **Mateo:** cambios del worker en código (`ShutdownTimeout`, salud, exportador de la cola) antes de la VM.
- **Daniela:** traspaso del Escenario 1 a Samara y pantallas de catálogo y detalle de curso (sus endpoints ya existen).

La documentación (arquitectura, informe de capacidad, deuda técnica) y el video se arman al final, con todo desplegado y medido. Mientras tanto cada quien guarda sus evidencias (registros, fechas UTC, commits) para no reconstruirlas después.

10. Repartición del frontend

Cada persona hace las pantallas del módulo que construyó en el backend, así conoce los endpoints y las reglas de negocio. Para no pisarse en el código, cada quien trabaja en su propia carpeta y las piezas compartidas son componentes con un contrato fijo.

10.1 Base común (Samara, primero)

- Carpeta `frontend/` en el repositorio: React + Vite + TypeScript, `react-router` y `hls.js`.
- **Cliente HTTP único** (`src/api/cliente.ts`): rutas relativas `/api/v1/...`, `credentials: 'include'` y encabezado `X-CSRF-Token` con el valor de la cookie `mooc_csrf` en POST, PUT, PATCH y DELETE. Nadie usa `fetch` directo.
- En desarrollo, `vite.config.ts` redirige `/api` a `localhost:8080`; en la nube el balanceador sirve frontend y API en el mismo dominio, así que no hace falta CORS.
- **Sesión:** `POST /auth/login`, `GET /me`, `POST /auth/logout`; contexto con el usuario y su rol; rutas protegidas por rol (un estudiante no entra a `/profesor/*`).
- **Componentes de estado** que todos reutilizan: Cargando, Vacío, Error, Éxito y Procesando. El enunciado exige mostrarlos.
- Carpetas por persona: `src/paginas/catalogo` y `src/paginas/profesor` (Daniela), `src/componentes/media` (Mateo), `src/paginas/estudiante` y `src/componentes/quiz` (Camila).

10.2 Pantallas y responsables

Persona	Ruta de la interfaz	Qué hace	Endpoints de la API
Samara	`/login`, menú, cierre de sesión	Iniciar y cerrar sesión, redirigir por rol, bloquear rutas ajenas	`POST /auth/login`, `GET /me`, `POST /auth/logout`
Daniela	`/catalogo`, `/cursos/:id`	Catálogo de cursos publicados, detalle e inscripción	`GET /catalog`, `GET /courses/:id`, `POST /courses/:id/enrollments`
Daniela	`/profesor/cursos`	Mis cursos (propios)	Endpoint nuevo (9.1, #3)
Daniela	`/profesor/cursos/nuevo`, `/profesor/cursos/:id/editar`	Crear y editar el curso con módulos, unidades y recursos; publicar y mostrar el resultado o los errores de las reglas	`POST /courses`, `POST /courses/:id/draft`, `PATCH /versions/:id`, módulos, unidades y recursos (POST/PATCH/DELETE), `POST /courses/:id/publish`, `GET /courses/:id/preview`
Mateo	Componente `<CargaArchivo>` dentro del editor	Subir archivos por partes con URL firmadas y mostrar el estado de procesamiento; un archivo en proceso no aparece como disponible	`POST /resources/:id/uploads/initiate`, `GET /uploads/:id/parts/:n/url`, `POST /uploads/:id/complete`, `GET /resources/:id/media`
Mateo	Componente `<VisorRecurso>`	Reproductor HLS y visor de PDF y documentos para el estudiante	`GET /resources/:id/playback`
Camila	`/mis-cursos`, `/aprender/:cursoId`	Mis cursos inscritos, navegación por módulos, unidades y recursos, registro y consulta del progreso	`GET /me/enrollments`, `POST /resources/:id/progress/open|heartbeat|complete`, `GET /courses/:id/progress`
Camila	`/quiz/:quizId`	Presentar el quiz y ver el resultado; doble envío sin doble calificación	`POST /quizzes/:id/attempts`, `PUT /attempts/:id/answers/:q`, `POST /attempts/:id/submit`, `GET /attempts/:id`
Camila	Componente `<EditorQuiz>` dentro del editor	Crear un quiz básico con preguntas y opciones	`POST /resources/:id/quiz`, `POST /quizzes/:id/questions`, `POST /questions/:id/options`, `GET /quizzes/:id/authoring`

10.3 Cómo se juntan las piezas

- El editor del curso de Daniela muestra, según el tipo de recurso, `<CargaArchivo resourceId>` de Mateo o `<EditorQuiz resourceId>` de Camila. La página de aprendizaje de Camila muestra `<VisorRecurso resource>` de Mateo.
- Los contratos (nombre del componente y sus props) se acuerdan el día que Samara suba la base; después cada quien trabaja sin esperar a los demás.
- Ramas `entrega3/<nombre>` que salen de `main` y pull request pequeños y frecuentes, para que la integración no quede toda para el final.
- **Recorrido conectado** (lo que se evalúa y se muestra en el video): profesor crea curso, sube video y PDF, crea quiz y publica; estudiante lo encuentra, se inscribe, ve el contenido, registra progreso y presenta el quiz. Camila lleva la matriz de verificación con rutas y evidencias.